INSTITUTO SUPERIOR DE TRANSPORTES E COMUNICAÇÕES

Licenciatura em Engenharia e Ciências dos Computadores — LECC 31

Sistemas Distribuídos e Paralelos

Teste 2 — Relatório Técnico

SCAD

Sistema de Compras Automáticas Distribuídas

Membros do grupo: Bill Arsenio Ndayisaba

Docente: MSc. Sérgio J. Sambo

Repositório: https://github.com/Bill-Ars/scad

Maputo, novembro de 2026

Índice

1. Introdução e objetivos
2. Arquitetura do sistema
    2.1 Fluxo de uma compra automática
3. Sistemas Operativos Distribuídos
    3.1 Exclusão mútua: algoritmo de Ricart-Agrawala
    3.2 Sincronização de relógios: algoritmo de Berkeley
4. Sistemas de Bases de Dados Distribuídos
    4.1 Modelo de dados e alocação
    4.2 Fragmentação horizontal
    4.3 Replicação mestre-escravo
    4.4 Consistência: Two-Phase Commit
5. Sistema de Ficheiros Distribuído
6. Java / CORBA
7. Serviços Web Distribuídos
8. Testes de tolerância a falhas
9. Limitações e trabalho futuro
10. Conclusão
Anexo A — Deploy

1. Introdução e objetivos

O presente relatório descreve o desenho e a implementação do **SCAD — Sistema de Compras Automáticas Distribuídas**. O sistema monitoriza os níveis de stock de três armazéns regionais (Maputo, Beira e Nampula) e, de forma automática e descentralizada, emite ordens de compra a fornecedores externos sempre que um produto atinge o stock mínimo.

O objetivo principal é aplicar, num sistema funcional, os conceitos fundamentais de sistemas distribuídos exigidos no enunciado: exclusão mútua e sincronização de relógios (SOD — Sistemas Operativos Distribuídos), fragmentação, replicação e consistência transacional (SBDD — Sistemas de Bases de Dados Distribuídos), armazenamento partilhado tolerante a falhas (DFS — Distributed File System), comunicação por objetos distribuídos (CORBA — Common Object Request Broker Architecture) e integração por serviços web (REST — Representational State Transfer, e SOAP — Simple Object Access Protocol).

O sistema foi implementado em Java (Spring Boot 4 sobre Java 21 e JacORB sobre Java 8), PostgreSQL 16 e Apache Hadoop 3, e é executado em 21 contentores Docker orquestrados por Docker Compose. O código e as instruções de deploy encontram-se no repositório indicado na capa.

2. Arquitetura do sistema

Cada nó do sistema é um contentor Docker com processo, sistema de ficheiros e endereço IP próprios, ligado aos restantes apenas pela rede virtual `scad-net`. Esta propriedade permite tratar cada contentor como um nó independente, que comunica exclusivamente por mensagens e que pode falhar isoladamente (`docker compose stop`). A Figura 1 apresenta a arquitetura.

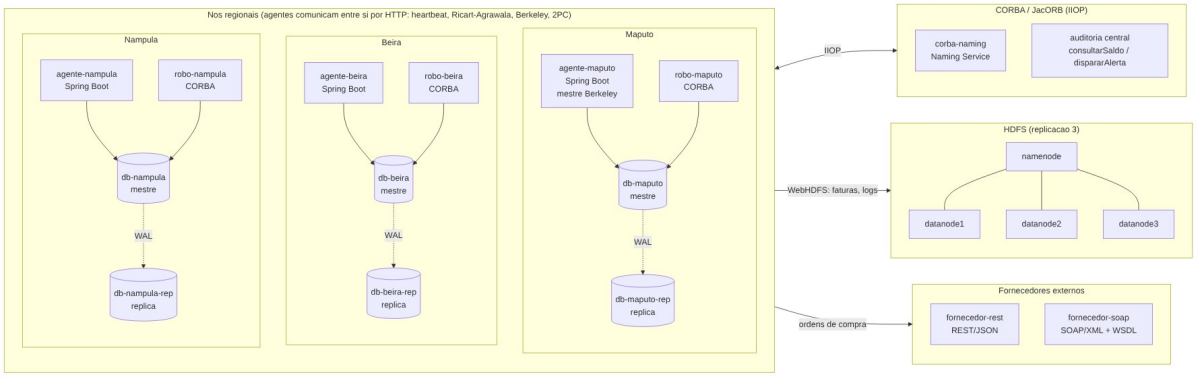


Figura 1 — Arquitetura do SCAD (21 contentores numa rede Docker)

Camada	Serviços	Tecnologia
Agentes regionais	`agente-maputo`, `agente-beira`, `agente-nampula`	Spring Boot 4.1, Java 21
Base de dados distribuída	3 mestres (`db-*`) e 3 réplicas (`db-*-rep`)	PostgreSQL 16, replicação por streaming
Comunicação interna legada	`corba-naming`, `auditoria`, `robo-*`	JacORB 3.9, Java 8, IIOP
Sistema de ficheiros	`namenode`, `datanode1..3`	Apache Hadoop 3 (HDFS), WebHDFS

Camada	Serviços	Tecnologia
Fornecedores externos	`fornecedor-rest`, `fornecedor-soap`	Spring Boot, JSON, SOAP 1.1/WSDL

O mesmo código de agente é executado três vezes; a região, o desvio de relógio, a base de dados e os pares de cada nó são definidos por variáveis de ambiente. Os nós encontram-se pelo nome (DNS interno do Docker), o que torna a localização transparente.

2.1 Fluxo de uma compra automática

A Figura 2 resume a interação entre componentes numa compra automática, que integra as cinco componentes do enunciado:

1. O agente deteta, no seu fragmento local, que um produto está abaixo do mínimo; em paralelo, o robô de stock regional envia um alerta à auditoria central por CORBA.
2. O agente pede a secção crítica do lote `fornecedor:produto` com o algoritmo de Ricart-Agrawala.
3. Dentro da secção crítica, verifica se outro nó já encomendou o lote; em caso afirmativo, não duplica a compra.
4. Reconstrói o stock global e calcula as quantidades a repor em todas as regiões abaixo do mínimo.
5. Regista a ordem em todos os armazéns envolvidos de forma atómica, com Two-Phase Commit.
6. Envia a ordem ao fornecedor pela rota REST/JSON ou SOAP/XML, conforme o tipo do fornecedor.
7. Guarda a fatura PDF no HDFS e regista a auditoria com a hora sincronizada pelo algoritmo de Berkeley.

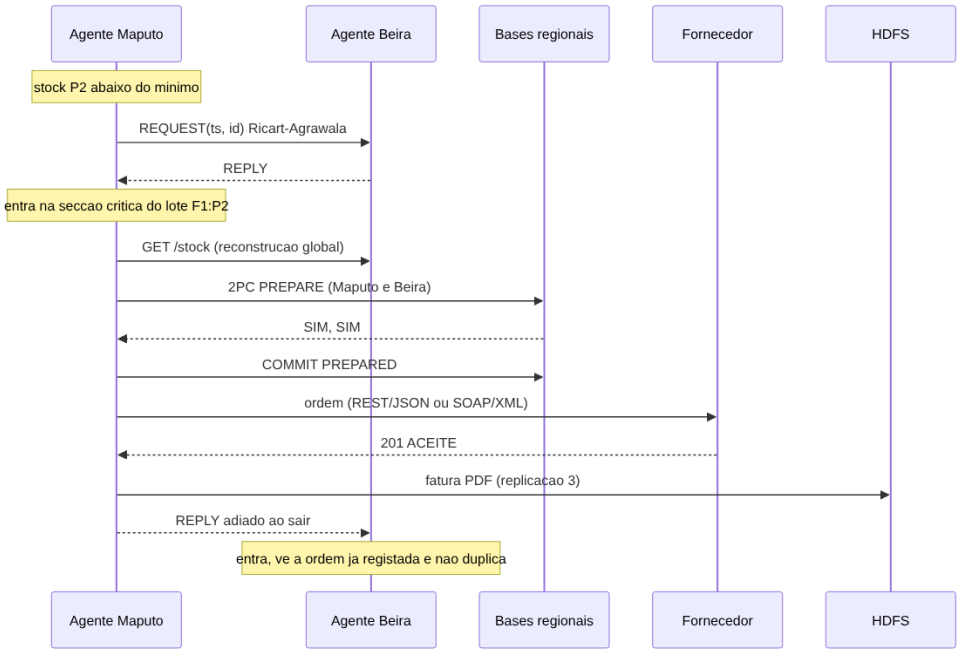


Figura 2 — Sequência de uma compra automática com dois nós em concorrência

Resultado obtido. Com Maputo e Beira a detetar ao mesmo tempo a falta do produto P2 (fornecedor F1), Maputo entrou primeiro na secção crítica (59 ms), reconstruiu o stock global (Maputo 50 e Beira 45 unidades a repor), executou o 2PC (votos SIM, SIM e decisão

COMMIT), enviou uma única ordem de 95 unidades pela rota REST e guardou a fatura no HDFS com replicação 3. A Beira entrou na secção crítica 846 ms depois, encontrou a ordem já registada no seu fragmento e não duplicou a compra. Num segundo cenário, Nampula comprou o produto P4 ao fornecedor F2 pela rota SOAP. Todos os eventos ficaram na auditoria com a hora sincronizada.

3. Sistemas Operativos Distribuídos

3.1 Exclusão mútua: algoritmo de Ricart-Agrawala

Decisão. Entre os algoritmos sugeridos (Ricart-Agrawala e Anel), foi escolhido o **Ricart-Agrawala** por três razões: não depende de um testemunho que se possa perder, atende os pedidos pela ordem em que foram feitos (justiça) e o seu custo, 2(N−1) mensagens por entrada, é baixo para três nós (4 mensagens).

Implementação. Cada recurso tem uma secção crítica própria, identificada pelo par `fornecedor:produto` (por exemplo, `F1:P2`), para que compras de lotes diferentes não se bloqueiem. Cada nó mantém um relógio lógico de Lamport e um estado por recurso (LIVRE, A_PEDIR ou DENTRO):

- para entrar, o nó incrementa o relógio e envia `REQUEST(ts, id)` aos restantes nós;
- ao receber um pedido, responde de imediato se estiver LIVRE ou se o pedido recebido for mais antigo; caso contrário adia o REPLY. Os empates de timestamp são resolvidos pelo identificador do nó (Maputo = 1, Beira = 2, Nampula = 3);
- entra quando recebe REPLY de todos os nós vivos e, ao sair, envia os REPLY adiados.

Tolerância a falhas. O algoritmo original bloqueia se um nó falhar, porque nunca recebe o seu REPLY. Para o evitar, o algoritmo usa o detetor de falhas por heartbeat (pedido `/estado` a cada 5 s com timeout de 2 s): os nós dados como em falha são retirados da espera, e existe ainda um timeout global de 20 s.

Resultado obtido. Com Maputo e Beira a pedir o lote `F1:P2` em simultâneo, ambos com `ts = 1`, o empate foi resolvido pelo identificador: Maputo entrou primeiro e registou `ADIA o REPLY a agente-beira (estado=DENTRO)`; a Beira só entrou quando Maputo saiu, cerca de 5 s depois. Com o agente de Nampula desligado, a exclusão mútua continuou a funcionar e Nampula foi ignorado por estar em falha.

3.2 Sincronização de relógios: algoritmo de Berkeley

Decisão. Todos os contentores partilham o relógio do mesmo kernel; com NTP (Network Time Protocol) não haveria desvio para corrigir nem resultado observável. Optou-se pelo **algoritmo de Berkeley**, implementado em Java, em que cada nó tem um desvio simulado (Maputo 0 s, Beira +25 s, Nampula −10 s).

Funcionamento. O agente de Maputo é o mestre (time daemon). Em cada ronda: pede a hora a cada nó; compensa metade do tempo de ida e volta (RTT); calcula a diferença de cada relógio; ignora diferenças superiores a 10 minutos; calcula a média e envia a cada nó o **ajuste relativo** (média − diferença), que se mantém correto independentemente do atraso da mensagem.

Nó	Desvio inicial	Diferença	Ajuste	Desvio final medido
Maputo (mestre)	0 ms	0 ms	+5 000 ms	5 000 ms
Beira	+25 000 ms	+25 000 ms	−20 000 ms	5 001 ms
Nampula	−10 000 ms	−10 000 ms	+15 000 ms	5 000 ms

Tabela 1 — Ronda de Berkeley: média das diferenças = +5 000 ms

Todas as transações e auditorias passam a ser registadas com a hora corrigida. A diferença residual de 1 ms deve-se ao tempo de rede. O mestre é fixo; a eleição de um novo mestre (por exemplo, pelo algoritmo Bully) fica como trabalho futuro.

4. Sistemas de Bases de Dados Distribuídos

4.1 Modelo de dados e alocação

Tabela	Alocação	Conteúdo
`armazem`, `produto`, `fornecedor`	Replicadas integralmente nos 3 nós	Dados de referência, pouco alterados e muito lidos
`stock`	Fragmentada por região	produto, região, quantidade, limite mínimo, fornecedor
`ordem_compra`	Fragmentada por região	ordem, produto, fornecedor, quantidade, estado, data
`auditoria`	Local a cada nó	nó de origem, hora (Berkeley), operação, detalhe

Tabela 2 — Modelo da base de dados distribuída

4.2 Fragmentação horizontal

A tabela de stock é fragmentada por região: STOCK_MAPUTO = σ(regiao = 'Maputo') (STOCK), e o mesmo para a Beira e Nampula. A tabela global obtém-se pela união dos três fragmentos. As três regras de correção foram verificadas:

- **Disjunção**, garantida pela própria base de dados com `CHECK (regiao = 'Maputo')`: a inserção de uma linha da Beira no fragmento de Maputo foi recusada com o erro SQLSTATE 23514 (`check_violation`);
- **Reconstrução e completude**: o endpoint `/stock/global` devolveu as 12 linhas esperadas (4 produtos × 3 regiões);
- **Falha parcial**: com o agente da Beira desligado, a consulta global devolveu 8 linhas e indicou o fragmento indisponível, mantendo o sistema disponível.

4.3 Replicação mestre-escravo

Cada região tem um servidor mestre e uma réplica. A réplica é criada com `pg_basebackup` e recebe continuamente o WAL (Write-Ahead Log) do mestre por streaming, ficando disponível só para leitura (hot standby). A replicação é assíncrona: tem menor latência, com o risco de perder as últimas escritas se o mestre falhar.

O failover é feito no cliente: o endereço JDBC lista os dois servidores com `targetServerType=preferPrimary`. Nos testes, a réplica mostrou o estado `streaming` em `pg_stat_replication`; uma alteração no mestre foi visível na réplica; a réplica recusou

escritas; com o mestre da Beira parado, o agente passou automaticamente para `db-beira-rep` em modo só leitura e, após `pg_promote()`, voltou a aceitar escritas. Foi também observado o problema de **split-brain** quando o antigo mestre volta sem ser reconstruído como réplica, o que justifica a regra operacional de não o reativar diretamente.

4.4 Consistência: Two-Phase Commit

Decisão. O enunciado admite 2PC ou Paxos/Raft. O 2PC é um protocolo de compromisso atómico que exige unanimidade, o que corresponde ao requisito: uma compra multi-armazém fica registada em todos os armazéns envolvidos ou em nenhum. Paxos e Raft são protocolos de consenso por maioria, adequados à replicação de um log, não à atomicidade entre fragmentos distintos.

Implementação. O agente que inicia a compra é o coordenador; os agentes das regiões envolvidas são participantes. Na fase 1, cada participante insere a sua parte e executa `PREPARE TRANSACTION`, que grava a transação em disco (voto SIM); qualquer erro provoca `ROLLBACK` (voto NÃO), e a ausência de resposta conta como NÃO. O coordenador grava a decisão na auditoria **antes** da fase 2 e envia `COMMIT PREPARED` ou `ROLLBACK PREPARED`. Um endpoint de recuperação lê a decisão gravada e conclui a fase 2.

Teste	Situação	Resultado
1	Maputo e Beira votam SIM	COMMIT; a mesma ordem existe nas duas bases
2	Beira vota NÃO (quantidade 0 viola CHECK)	ABORT; Maputo desfaz a sua parte
3	Agente da Beira desligado	NÃO por ausência de resposta; ABORT
4	Coordenador falha após a decisão	Transações presas em `pg_prepared_xacts` (bloqueio); a recuperação lê COMMIT do log e conclui nas duas bases

Tabela 3 — Testes do 2PC

5. Sistema de Ficheiros Distribuído

Decisão. Entre as opções do enunciado, o NFS (Network File System) tem um único servidor, logo não tolera a sua queda; o GlusterFS exige módulos de kernel e contentores privilegiados, pouco viáveis em Docker Desktop. Foi escolhido o **HDFS** (Hadoop Distributed File System) com 1 NameNode e 3 DataNodes e **fator de replicação 3**.

O NameNode guarda os metadados e os DataNodes os blocos; cada bloco é copiado em pipeline para os três DataNodes. Os agentes acedem por WebHDFS: o NameNode indica o DataNode e os dados seguem diretamente para ele. São guardados no DFS as faturas PDF (`/scad/faturas`), os logs de erro partilhados (`/scad/logs/erros.log`) e os relatórios de auditoria.

Resultados. O `hdfs fsck` mostrou a fatura com `Live_repl=3` e estado HEALTHY; a fatura escrita por Maputo foi lida pela Beira e listada por Nampula (transparência de acesso e de localização); com o agente de Maputo e o datanode1 desligados, a Beira continuou a ler a fatura; dois nós acrescentaram linhas ao mesmo ficheiro de log. O NameNode é um ponto único de falha (os dados não se perdem, mas ficam inacessíveis); a configuração de alta disponibilidade fica fora do âmbito.

6. Java / CORBA

A comunicação interna entre o módulo central de auditoria e os robôs de stock regionais usa **JacORB 3.9**. O JacORB foi executado sobre Java 8, a plataforma para a qual esta versão foi desenvolvida; o resto do sistema usa Java 21. A interface IDL (Interface Definition Language) é a seguinte:

```
// ================================================================
//  SCAD - Interface IDL da comunicacao interna (CORBA / JacORB)
//  Modulo central de auditoria  <->  robos/agentes de stock locais
// ================================================================
module SCAD {

    // Alerta enviado por um robo quando um produto fica abaixo do stock minimo
    struct AlertaStock {
        string    produtoId;
        string    regiao;
        long      quantidadeAtual;
        long      limiteMinimo;
        long long timestamp;          // milissegundos desde 1970 (UTC)
    };

    typedef sequence<AlertaStock> ListaAlertas;

    exception ProdutoInexistente { string motivo; };

    // Implementado por cada robo de stock regional; invocado pela auditoria
    interface AgenteStock {
        readonly attribute string regiao;
        long consultarSaldo(in string produtoId) raises (ProdutoInexistente);
    };

    // Implementado pelo modulo central de auditoria; invocado pelos robos
    interface Auditoria {
        oneway void dispararAlerta(in AlertaStock alerta);   // sem resposta: o robo nao
bloqueia
        ListaAlertas listarAlertas();
    };
};
```

O compilador IDL do JacORB gera, a partir deste ficheiro, os stubs (cliente), os skeletons POA (Portable Object Adapter, servidor), os helpers e os holders. A arquitetura é:

- `corba-naming`: Naming Service do JacORB, cuja referência (IOR — Interoperable Object Reference) é partilhada por um volume;
- `auditoria`: servant de `Auditoria`, registado como "Auditoria";
- `robo-*`: cada robô é servidor de `AgenteStock` (registado como "AgenteStock-<Região>") e cliente da auditoria; de 10 em 10 s verifica o stock e invoca `dispararAlerta`, declarada `oneway` para que o robô não bloqueie à espera da auditoria.

Resultados. Os quatro objetos registaram-se no Naming Service; os robôs dispararam alertas para P2 (Maputo e Beira) e P4 (Nampula); ao baixar o stock de P1 em Nampula, o alerta chegou à auditoria em menos de 10 s; `consultarSaldo` respondeu em 40 a 100 ms por chamada IIOP (Internet Inter-ORB Protocol); com o robô da Beira desligado, a chamada falhou com a exceção `TRANSIENT` e os restantes robôs continuaram a responder.

7. Serviços Web Distribuídos

O envio final da ordem ao fornecedor tem duas rotas, escolhidas pelo campo `tipo_integracao` da tabela `fornecedor`:

Aspeto	REST/JSON (fornecedor moderno)	SOAP/XML (fornecedor legado)
Endpoint	`POST /api/ordens`	`POST /ws` (envelope SOAP 1.1)
Contrato	Recursos e códigos HTTP	WSDL em `/ws?wsdl` (document/literal, XSD)
Sucesso	201 Created (nova); 200 OK (repetida)	`EnviarOrdemResponse` com estado ACEITE
Erro de pedido	400 Bad Request	`soap:Fault` com faultcode `soap:Client`

Tabela 4 — Rotas de integração com fornecedores

Robustez. O cliente usa timeouts (2 s de ligação, 3 s de leitura), faz até 3 tentativas com espera crescente (1 s, 2 s) em falhas de rede ou erros 5xx, não repete pedidos inválidos (4xx ou `soap:Client`) e garante **idempotência** pelo identificador da ordem: o fornecedor que já conhece o `ordemId` devolve a resposta original em vez de criar outra ordem. Os documentos XML são lidos com DTD desativado, para prevenir ataques XXE (XML External Entity).

Resultados. O WSDL foi publicado; a mesma ordem REST devolveu 201 e depois 200; a quantidade 0 devolveu 400 em REST e `SOAP FAULT` em SOAP, sem novas tentativas; com o fornecedor REST desligado, houve 3 tentativas falhadas e, após a recuperação, o reenvio da mesma ordem devolveu 201 e uma segunda repetição devolveu 200, confirmando que a ordem não foi duplicada.

8. Testes de tolerância a falhas

Falha simulada	Comando	Comportamento observado
Agente regional	`docker compose stop agente-beira`	Heartbeat deteta EM FALHA em menos de 5 s; Ricart-Agrawala ignora o nó; 2PC aborta
Mestre de uma região	`docker compose stop db-beira`	Leituras continuam na réplica; `pg_promote()` repõe as escritas
Coordenador do 2PC	falha simulada entre fases	Participantes bloqueados; recuperação pelo log da decisão
DataNode e agente escritor	`stop agente-maputo datanode1`	Fatura continua legível a partir da Beira
Robô de stock	`docker compose stop robo-beira`	`TRANSIENT` para esse robô; restantes respondem
Fornecedor	`docker compose stop fornecedor-rest`	3 tentativas, falha registada, reenvio sem duplicação

Tabela 5 — Resumo dos testes de resiliência

Todos os testes podem ser repetidos com os scripts da pasta `testes/` do repositório.

9. Limitações e trabalho futuro

- Todos os nós correm numa só máquina física: é simulada a falha de processos, não a falha do anfitrião. A configuração permite mover um nó para outro computador alterando apenas endereços.
- O coordenador do 2PC e o NameNode do HDFS continuam a ser pontos únicos de falha; o 3PC, um protocolo de consenso ou o HDFS em alta disponibilidade seriam a evolução natural.
- O mestre de Berkeley é fixo e o failover da base de dados exige a promoção manual da réplica.
- As ordens recebidas pelos fornecedores simulados são mantidas em memória.

10. Conclusão

O SCAD cumpre as cinco componentes do enunciado com mecanismos efetivamente implementados e testados: exclusão mútua por Ricart-Agrawala e sincronização por Berkeley; fragmentação horizontal em três nós com replicação mestre-escravo e atomicidade por 2PC; armazenamento de faturas e logs em HDFS com replicação 3; comunicação interna por CORBA/JacORB com uma IDL própria; e integração com fornecedores por REST/JSON e SOAP/XML. Os testes mostraram que a falha de um nó (agente, base de dados, DataNode, robô ou fornecedor) não compromete nem a consistência dos dados nem a disponibilidade do resto do sistema, e evidenciaram também as limitações clássicas destes protocolos, como o bloqueio do 2PC e o split-brain na replicação.

Anexo A — Deploy

```
git clone https://github.com/Bill-Ars/scad.git
cd scad
docker compose up -d --build
docker compose ps
powershell -ExecutionPolicy Bypass -File .\testes\teste-integracao.ps1
```